

1. malloc 与 new 的区别

malloc 和 new 都是用于在堆上动态分配内存，但它们存在本质区别。malloc 是C语言的标准库函数，它只负责分配指定大小的字节空间，返回一个 void* 类型的指针，不具备类型检查，且需要手动计算所需空间大小。分配成功后，它不会调用对象的构造函数。与之对应，free 函数仅负责释放内存，不调用析构函数。

new 是C++的运算符，它在分配内存时会根据指定的类型自动计算空间大小，并返回对应类型的指针，具有类型安全性。更重要的是，new 在分配内存后会自动调用对象的构造函数进行初始化。同样，delete 运算符在释放内存前会先调用对象的析构函数，完成资源的清理。此外，new 在内存分配失败时，默认会抛出 std::bad_alloc 异常，而 malloc 则返回 NULL。

2. 单例模式

单例模式是一种常用的设计模式，其核心思想是确保一个类在整个应用程序中只有一个实例，并提供一个全局的、唯一的访问点来获取这个实例。

常用于管理共享资源，如数据库连接池、日志记录器或全局配置对象。实现单例模式的关键在于将类的构造函数设为私有，以防止外部直接创建实例，同时提供一个公共的静态方法（如 getInstance()）来创建或返回那个唯一的实例。

3. 饿汉式与懒汉式单例

单例模式主要有两种实现方式：饿汉式和懒汉式。

- **饿汉式**：在程序启动或类加载时就立即创建实例。这种方式实现简单，并且由于实例在多线程访问之前就已经创建好了，因此是**线程安全**的；它的缺点是，无论后续是否会使用到这个实例，都会在程序启动时占用内存资源，可能导致启动速度变慢；
- **懒汉式**：在第一次调用 getInstance() 方法时才创建实例，实现了“延迟加载”。这种方式可以节省启动时的内存开销。但它的缺点是在多线程环境下，如果不进行同步控制，可能会有多个线程同时进入创建实例的代码块，导致创建出多个实例，破坏了单例的唯一性；因此，懒汉式模式必须进行**线程安全处理**，常见的处理方式有加锁（如 std::mutex）或利用C++11及以后版本中静态局部变量初始化的线程安全特性；

STL 容器

4. vector 与 list 的区别

std::vector 和 std::list 是两种常用的序列容器；

- vector 的底层是**动态数组**，其内存存在物理上是**连续**的。它支持高效的随机访问，通过下标访问元素的时间复杂度为 $O(1)$ ，但在容器中间或开头进行插入和删除操作时，需要移动后续所有元素，效率较低，时间复杂度为 $O(n)$ ；
- list 的底层是**双向链表**，其内存是**不连续**的，每个节点除了存储数据外，还包含指向前后节点的指针。这使得它不支持随机访问，访问某个元素需要从头或尾遍历，时间复杂度为 $O(n)$ ；但它的优势在于，在任何位置进行插入和删除操作都非常高效，只需要修改相邻节点的指针即可，时间复杂度为 $O(1)$ ；

5. vector 的扩容与缩容机制

vector 为了保持内存的连续性，在空间不足时会自动进行扩容。当向 vector 中添加元素导致其容量（capacity）不足时，它会分配一块更大的新内存空间（通常是原容量的1.5倍或2倍），然后将所有旧元素拷贝到新空间，最后释放旧空间。这个过程虽然开销较大，但通过“预留空间”的策略，摊还后的时间复杂度仍为 $O(1)$ ；

vector **没有自动的缩容机制**；即使执行 `pop_back()` 或 `clear()` 等操作删除了大量元素，vector 的容量 `capacity()` 也不会自动减小；这是为了避免在频繁增删元素时反复进行内存分配和释放；

如果确实需要回收多余的内存，可以采用 **`shrink_to_fit()`** 方法（C++11），它会请求容器将容量减少到与大小 `size()` 相匹配，或者使用“copy-and-swap”的技巧：`std::vector(my_vector).swap(my_vector);`，通过创建一个临时的、大小刚好的 vector 并与之交换内容，来达到缩减内存占用的目的；

计算机网络

6. TCP 与 UDP 的区别

TCP（传输控制协议）和UDP（用户数据报协议）是传输层的两个核心协议，它们的主要区别在于连接性、可靠性和效率；

- **TCP 是面向连接的、可靠的**协议；在数据传输前，必须通过“三次握手”建立连接。它提供了一系列机制来保证数据传输的可靠性，包括序列号、确认应答、超时重传、流量控制和拥塞控制。因此，TCP适合对数据准确性要求高的应用，如文件传输（FTP）、网页浏览（HTTP）、电子邮件（SMTP）等；
- **UDP 是无连接的、不可靠的**协议；它在发送数据前不需要建立连接，直接将数据报文（Datagram）发送出去，是一种“尽力而为”的服务；UDP不保证数据包的顺序、不保证不丢失、也不保证不重复；但正因如此，它的开销小、传输效率高、延迟低；UDP适合对实时性要求高、能容忍少量丢包的应用，如视频会议、在线直播、DNS查询等；

7. TCP 的可靠性保障

首先，它通过**序列号**对发送的每个字节进行编号，接收方根据序列号来重组数据并检测丢失的包；

其次，**确认应答（ACK）机制**要求接收方每收到一个数据包，都要发送一个确认信息；如果发送方在一定时间内没有收到确认，就会触发**超时重传**机制，重新发送丢失的数据包；

此外，**流量控制**利用滑动窗口协议，防止发送方过快地发送数据而导致接收方缓冲区溢出；

拥塞控制机制则根据网络状况动态调整发送速率，避免造成网络拥堵；

校验和机制也用于检测数据在传输过程中是否出错；

8. 三次握手的详细流程

TCP 的三次握手是建立连接的过程，旨在同步双方的序列号并确认双方的收发能力都正常

1. **第一次握手**：客户端向服务器发送一个特殊的报文段，其中同步位 SYN 被置为1，并随机选择一个初始序列号 `seq=x`；此时，客户端进入 SYN_SENT 状态；
2. **第二次握手**：服务器收到客户端的 SYN 报文后，如果同意建立连接，会回复一个报文段。该报文段中，同步位 SYN 和确认位 ACK 都被置为1；服务器会确认客户端的序列号，设置确认号 `ack=x+1`，同时自己也选择一个初始序列号 `seq=y`；此时，服务器进入 SYN_RECV 状态
3. **第三次握手**：客户端收到服务器的 SYN+ACK 报文后，会再发送一个确认报文段。该报文段中，ACK 位被置为1，确认号 `ack=y+1`，自己的序列号则为 `seq=x+1`；这个报文发送完毕后，客户端和服务器都进入 ESTABLISHED 状态，连接建立成功；

9. 解决 TCP 粘包问题的四种方法

TCP 是一个面向字节流的协议，它不保留应用层消息的边界，因此可能发生“粘包”（多个小包合并成一个大包）或“半包”（一个大包被拆分成多个小包）问题，解决这个问题通常有四种方法：

1. **固定长度消息**：发送方和接收方约定每条消息都使用固定的长度。接收方每次都读取该长度的字节作为一个完整的消息。这种方法简单但灵活性差；
2. **特殊分隔符**：在每条消息的末尾添加一个特殊的分隔符（如换行符 \n）。接收方通过扫描这个分隔符来切分消息。这种方法适用于文本协议；
3. **消息头包含长度**：这是最常用和最灵活的方法。在每条消息前附加一个固定长度的消息头，消息头中包含一个字段，用于描述紧随其后的消息体的长度。接收方先读取固定长度的头部，解析出消息体的长度，然后再读取相应长度的字节作为一条完整的消息；
4. **短链接**

数据库与 Qt

10. 数据库存储过程与函数的区别

存储过程和函数都是预先编译好并存储在数据库中的一组 SQL 语句集合，但它们在用途和行为上有所不同；

- **返回值**：这是最主要的区别。**函数必须有一个返回值**，且只能有一个；而**存储过程可以没有返回值，也可以通过输出参数（OUT 或 INOUT）返回多个值**；
- **调用方式**：函数可以直接在 SELECT 等 SQL 语句中像普通函数一样调用，并作为表达式的一部分；存储过程则需要通过 CALL 或 EXECUTE 语句来独立执行；
- **用途**：函数通常用于执行计算并返回一个单一的结果，功能较为集中；存储过程则更强大，可以执行一系列复杂的数据库操作，如批量更新、数据迁移等，实现复杂的业务逻辑；

11. Qt connect 函数的第五个参数

Qt 中 connect 函数的第五个参数用于指定信号和槽之间的**连接类型（Qt::ConnectionType）**

- Qt::AutoConnection (默认值): 如果信号的发送者和接收者在同一个线程，则行为如同 DirectConnection；如果不在同一个线程，则行为如同 QueuedConnection；
- Qt::DirectConnection: 槽函数在信号发射的线程中被**直接同步调用**，槽函数会立即执行，就像一个普通的函数调用；
- Qt::QueuedConnection: 槽函数会被**异步调用**；当信号发射时，Qt 会将这个调用封装成一个事件，并将其放入接收者所在线程的事件队列中；槽函数将在接收者线程的事件循环下一次处理事件时被执行。这是实现线程间安全通信的关键；
- Qt::BlockingQueuedConnection: 与 QueuedConnection 类似，但它会**阻塞**信号发射的线程，直到槽函数执行完毕；
- Qt::UniqueConnection: 行为与 AutoConnection 类似，但可以防止重复连接；

12. 为什么要用 SDL

SDL (Simple DirectMedia Layer) 是一个跨平台的、开源的多媒体开发库；在项目中选择使用 SDL：

- **跨平台能力**：SDL 提供了对音视频、输入设备（键盘、鼠标、游戏手柄）等的统一抽象接口，使得编写一次代码就可以在多个平台上编译运行，简化了跨平台开发；

- **底层访问与高性能**：SDL 提供了对图形硬件（通过 OpenGL, Direct3D, Vulkan 等）和音频设备的底层、直接的访问能力；非常适合需要高性能渲染和低延迟音频处理的应用，如游戏开发；
- **简洁易用**：相比于直接使用平台特定的API（如 Win32 API, X11），SDL 的接口更加简洁和易于上手，开发者可以更专注于应用逻辑本身，而不是处理复杂的平台细节；在音视频播放场景中，SDL与 FFmpeg 等解码库结合使用，可以方便地实现一个跨平台的播放器；

数组和指针的区别

数组和指针是两个截然不同的概念，但它们在使用时，尤其是在作为函数参数时，表现出相似性；

- **本质与内存**：**数组**是一块连续的内存空间，用于存储一系列相同类型的元素，数组名代表的是这整块内存空间的首地址，其大小在编译时确定；**指针**是一个变量，它存储的是另一个变量的内存地址**，指针本身的大小（通常是4或8字节）与它指向的数据类型无关；
- **可修改性**：数组名是一个**常量指针**，它指向数组首地址，其值不可被修改（例如，arr++ 是非法的）。而指针是一个变量，可以被修改以指向不同的内存地址；
- **sizeof 运算符**：sizeof(数组名) 返回的是整个数组所占用的总字节数。sizeof(指针) 返回的是指针变量自身的大小；当数组作为函数参数传递时，它会**退化为指针**，此时在函数内部 sizeof 这个参数得到的是指针的大小，而不是原数组的大小；

引用和指针的区别

引用可以看作是变量的一个“别名”，而指针存储的是变量的地址；

1. **初始化与空值**：引用在定义时**必须被初始化**，且一旦绑定到一个对象，就不能再引用其他对象；引用不能为空；**指针**可以不初始化（会成为野指针），也可以在任何时候指向不同的对象或 nullptr；
2. **语法与解引用**：使用引用时，语法上就像操作原始变量本身，无需解引用；使用指针则需要 * 运算符来解引用，访问其指向的数据；
3. **内存占用**：sizeof(引用) 得到的是其所引用对象的大小，而 sizeof(指针) 得到的是指针变量自身的大小；

野指针和悬挂指针

- **野指针 (Wild Pointer)**：指**未被初始化**的指针；由于它没有一个确定的、合法的指向，其值是随机的。解引用野指针会导致未定义的行为，通常会造成程序崩溃；
- **悬挂指针 (Dangling Pointer)**：指一个指针**曾经指向一个有效的对象，但该对象已经被释放或销毁**（例如，函数返回了局部变量的地址，或 delete 了一个指针后未将其置为 nullptr）；此时指针仍然保留着那个旧的内存地址，但该地址上的数据已经不再有效；解引用悬挂指针同样会导致未定义的行为；**避免悬挂指针的常用方法是，在 delete 指针后立即将其赋值为 nullptr**；

const char* p 与 char* const p 的区别

这是一个经典的 const 和指针结合的问题，关键在于 const 修饰的是谁；

- **const char* p**（或 char const* p）：const 修饰的是 char，意味着 p 指向的**内容是不可修改的**（*p 不可改），但指针 p 本身是可修改的（p 可以指向其他地址）；这通常用于函数参数，表示函数不会通过该指针修改传入的字符串；
 - **char* const p**：const 修饰的是 p，意味着**指针 p 本身是不可修改的**，必须在定义时初始化，之后不能再指向其他地址；但它指向的内容是可以通过 p 来修改的（*p 可改）；
-

C++ 三大特性

1. **封装 (Encapsulation)**: 将数据（属性）和操作数据的函数（方法）捆绑在一个“类”中，并通过访问修置符（public, private, protected）控制对内部成员的访问，隐藏实现细节，只暴露必要的接口；
2. **继承 (Inheritance)**: 允许一个类（子类）获取另一个类（父类）的属性和方法，实现了代码的复用，并建立了类之间的层次关系；
3. **多态 (Polymorphism)**: 允许不同类的对象对同一消息做出不同的响应，即“一个接口，多种实现”；

静态多态、动态多态的实现方式

- **静态多态 (编译时多态)**: 在**编译阶段**就确定了调用哪个函数；主要通过**函数重载**和**模板**来实现。编译器根据函数的参数类型、数量或模板参数来决定具体调用哪个版本的函数；
- **动态多态 (运行时多态)**: 在**运行阶段**才根据对象的实际类型来确定调用哪个函数；主要通过**虚函数**和**继承**来实现；当通过基类指针或引用调用一个虚函数时，程序会通过对象的虚函数表（vtable）在运行时查找到正确的函数地址并执行，这个过程称为动态绑定；

static 在 C++ 中的用法

static 关键字在 C++ 中有多种用途，其核心意义是“与类或文件作用域相关，而非与对象实例或函数调用相关”

1. **静态局部变量**: 在函数内部声明的 static 变量，其生命周期贯穿整个程序运行期间，但作用域仍限于该函数。它只会被初始化一次，即使函数被多次调用；
2. **静态全局变量/函数**: 在全局作用域（文件作用域）中，static 修饰的变量或函数的作用域被限制在**当前源文件**内，使其成为内部链接，其他源文件无法访问；
3. **静态数据成员**: 在类中，static 修饰的数据成员是**所有对象共享**的，属于类本身，而不是某个特定的对象实例。它必须在类外进行定义和初始化；
4. **静态成员函数**: 在类中，static 修饰的成员函数也属于类本身，不与任何对象实例绑定。因此，它**没有 this 指针**，不能直接访问非静态成员，只能访问静态成员；

RAII 特性是什么？举个使用的例子？

RAII，即“资源获取即初始化”，是 C++ 中一种重要的资源管理和异常安全编程技术；其核心思想是，**在对象的构造函数中获取资源（如内存、文件句柄、锁等），并在对象的析构函数中释放资源**；

通过将资源的生命周期与对象的生命周期绑定，可以确保无论程序是正常执行结束还是因异常而中途退出，只要对象被正确销毁（例如，栈上的对象离开作用域时会自动销毁），其析构函数就一定会被调用，从而保证资源总能被正确释放，避免了资源泄漏；

最经典的例子就是智能指针 `std::unique_ptr` 和 `std::shared_ptr`；

```

void process_file(const std::string& filename) {
    // 使用 std::ifstream, 它就应用了 RAII
    std::ifstream file(filename); // 构造时打开文件, 获取资源
    if (!file.is_open()) {
        throw std::runtime_error("Failed to open file");
    }

    // ... 对文件进行操作 ...
    // 如果在这里发生异常, file 对象会被自动销毁

} // 函数结束时, file 对象离开作用域, 其析构函数被自动调用, 文件被关闭

```

在这个例子中, 我们不需要手动调用 `file.close()`; 无论函数是正常返回还是因异常退出, `ifstream` 对象的析构函数都会确保文件句柄被释放;

STL 容器有哪些? 容器适配器?

STL 容器主要分为三类:

1. 序列容器 (Sequence Containers): 元素按线性顺序排列

- `std::vector`: 动态数组
- `std::list`: 双向链表
- `std::deque`: 双端队列, 结合了 `vector` 和 `list` 的部分优点
- `std::array`: 固定大小的数组 (C++11)
- `std::forward_list`: 单向链表 (C++11)

2. 关联容器 (Associative Containers): 元素根据键值自动排序

- `std::map`: 键值对集合, 键唯一, 基于红黑树
- `std::set`: 键的集合, 键唯一, 基于红黑树
- `std::multimap`: 允许键重复的 `map`
- `std::multiset`: 允许键重复的 `set`

3. 无序关联容器 (Unordered Associative Containers): 元素无序, 基于哈希表

- `std::unordered_map`, `std::unordered_set`, `std::unordered_multimap`, `std::unordered_multiset`

容器适配器 (Container Adaptors): 不是独立的容器, 而是对已有序列容器进行封装, 提供了特定的接口

- `std::stack`: 栈 (后进先出), 默认基于 `std::deque` 实现;
- `std::queue`: 队列 (先进先出), 默认基于 `std::deque` 实现;
- `std::priority_queue`: 优先队列 (堆), 默认基于 `std::vector` 实现;

进程与线程的区别和联系

• 区别:

1. **资源分配:** 进程是操作系统进行资源分配的基本单位; 它拥有独立的地址空间、内存、文件句柄等资源;
2. **调度:** 线程是CPU进行调度的基本单位; 一个进程可以包含多个线程, 这些线程共享该进程的所有资源;
3. **开销:** 创建、销毁和切换进程的开销远大于线程, 因为涉及独立的资源管理;

4. **通信**: 进程间通信（IPC）较为复杂，需要专门的机制；线程间由于共享内存，通信非常方便，但也带来了数据同步的挑战；

- **联系**: 线程是进程的一部分，是“轻量级的进程”；一个进程至少有一个主线程；线程的存在依赖于进程，进程销毁时，其所有线程也随之销毁；

epoll 与多路I/O复用技术

epoll 是 Linux 下一种高效的多路I/O复用技术；它的作用是让一个线程能够同时监视成千上万个文件描述符（如网络连接），并在其中任何一个或多个变得“就绪”（可读/可写）时，高效地通知该线程；这使得用少量线程处理大量并发连接成为可能，是构建高性能网络服务器的核心；

多路I/O复用技术主要有三种：

1. **select**: 经典且跨平台，但有**1024个文件描述符的数量限制**，并且每次调用都需要将整个FD集合从用户空间拷贝到内核，返回后还需要**轮询**整个集合来找到就绪的FD，效率随FD数量增加而急剧下降；
2. **poll**: 解决了 select 的数量限制问题，但内存拷贝和轮询的低效问题依然存在；
3. **epoll**: Linux独有，彻底解决了前两者的问题。它通过 `epoll_ctl` 将FD注册到内核的**红黑树**中，避免了重复拷贝；当有FD就绪时，内核通过**回调机制**直接将就绪的FD放入一个**链表**，`epoll_wait` 只需返回这个链表即可，无需轮询；效率高，与监视的总FD数量无关，只与活跃的FD数量有关；

线程池是什么？多线程和线程池的区别？

线程池是一种线程使用模式；它预先创建好一定数量的线程并放入一个“池”中进行统一管理；当有任务需要执行时，不是创建一个新线程，而是从池中取出一个空闲线程来执行任务；任务执行完毕后，该线程并不会被销毁，而是返回池中等待下一个任务；

多线程和线程池的区别：

- **多线程**是一种**并发编程的概念或技术**，指在一个程序中同时运行多个线程；
- **线程池**是一种具体的**管理和使用线程的模式或工具**；

可以将它们的关系理解为：**线程池是多线程技术的一种高效、可控的应用**；相比于“来一个任务就创建一个新线程”的朴素多线程模型，线程池有以下作用和优点：

1. 通过复用已存在的线程，避免了频繁创建和销毁线程带来的系统开销；
2. 任务可以直接由池中的现成线程执行，省去了创建线程的等待时间；
3. 可以统一控制并发线程的数量，防止因线程过多而耗尽系统资源，使系统运行更加稳定；